Chapter 2
C Fundamentals

CSE 4107 : Structured Programming I
REYNEP

Hello World!

```c
// hello.c

#include <stdio.h>

int main(void) {
    printf("hello, world\n");
}
```



2.1.1. Writing a Simple Program : Compiling and Linking (page 1)

The code, from last page, we need to write it down into a file, say, we name it `hello.c`. Despite its brevity, getting `hello.c` to run is more involved than you might expect.

First, we need to create a file named `hello.c` containing the program (any text editor will do). The name of the file doesn't matter, you can name it anything, but the `.c` extension is often required by compilers.

Next, we've got to convert the program to a form that the machine can execute. For a C program, that usually involves **3 steps**:

1. `PreProcessing`
   - *The program is first given to a* **preprocessor**
   - **preprocessor** *obeys stuff that begin with* `#` *e.g.*
     - `#include` *is called a* **"directive"** / **"command"**
     - `#include <stdio.h>`
   - *A* **preprocessor** *is a bit like* **"an editor"**;
     - *it can add/***copy-paste***/concatenate things to the* `.c` *file and thus making* **modifications**.
     - `#include <stdio.h>`
       - **copy-pastes** *all the content of a file called* **"stdio.h"**

2. `Compiling`
   - *The preprocessor-***modified** `.c` *file now goes to a* **compiler**.
   - **Compiler** *translates our* `hello.c` *into* **"binary machine instructions"**/ `machine-code` / `object-code`.
     - *e.g.* `hello.c` ➡️ `hello.obj`
   - *However, The program isn't quite ready to run just yeet!*

3. `Linking`
   - *This is the final step & is required, if we wanna run our* `hello.c`.
   - *A* **linker** *combines the* `hello.obj` *with other* `.obj` *files e.g.*
     - *we used* `printf()`.
     - `printf()`*'s binary code is inside another* `.obj` *provided by our OS.*
     - **Linker** *links that binary code with* `hello.obj`
     - *thus yielding a complete executable program.*

2.1.2. Writing a Simple Program : Compiling and Linking (page 2)

Fortunately, this process is often automated, so you won't find it too onerous.
In fact, the preprocessor is usually integrated with the compiler, so you probably won't even notice it at work.

The commands necessary to compile and link vary, depending on the compiler and operating system. Under UNIX, the C compiler is usually named `cc`. To compile and link the `hello.c` program, enter the following command in a terminal or command-line window:

```
cc hello.c
    # PreProcessor is usually integrated with compiler
    # Linking is automatic when using cc; no separate link command is necessary.
```

After **compiling** and **linking** `hello.c`

- `cc` *leaves the* ***"compiled"*** `binary-code` *in a file named* `a.out` *by default.*
- `cc` *has many options, tho.*
  - *one of them* ➡️ `-o` *allows us to choose the name of the output file. e.g.*

```
cc hello.c -o hello         # Linux
cc hello.c -o hello.exe     # Windows, (altho, windows wouldnt have `cc` by default)
```

2.1.3. Writing a Simple Program : Compiling and Linking (page 3)

The GCC Compiler

One of the most popular C compilers is the `GCC compiler`, which is supplied with **Linux** but is available for many other platforms as well. Using this compiler is similar to using the traditional **UNIX** `cc` compiler. For example, to compile the `hello.c` program, we would use the following command:

```
gcc hello.c -o hello         # Linux
gcc hello.c -o hello.exe     # Windows (available via, `msys2` + `mingw64`)
```

Others

1. `cc` *: not a compiler itself, rather just a* ***"Environment Variable"***
2. `gcc` *: GNU C Compiler (Linux/Unix)*
3. `clang` *: Apple's C Compiler*
4. `msvc` *: Microsoft Visual C (only for Windows)*

2.1.4. Summary 🟪 `reynep's`

1. `PreProcessing`
   - `#include` = **"directive"** / **"command"** - *(starts with* `#` *)*
   - `#include <stdio.h>`
     - **copy-pastes** *all the content of a file called* **"stdio.h"**
2. `Compiling`
   - **Compiler** *translates* `hello.c` *into* **"binary machine instructions"**
     - *e.g.* `hello.c` ➡️ `hello.obj`
3. `Linking`
   - **linker** *combines the* `hello.obj` *with say* `stdio.obj`
4. `gcc`
   - `gcc` / `cc` / `clang` / `msvc` *does all 3 steps, then, after* **linking**, *hands us* `hello` / `hello.exe`

```
gcc hello.c -o hello         # Linux
gcc hello.c -o hello.exe     # Windows (available via, `msys2` + `mingw64`)
```

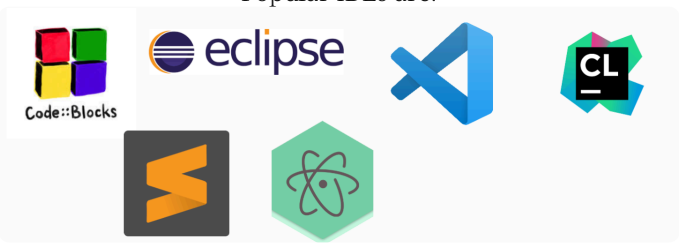
2.1.5. Writing a Simple Program : Integrated Development Environments

So far, we've assumed the use of `gcc` & `cc`. Those can only be used inside the `terminal` / `command-line`. But, we do have an better alternative.

- *The alternative is to use an integrated development environment (IDE)*
  - *IDE is A software package that allows us to*
    - **edit**
    - **compile**
    - **link**
    - **execute/run** &
    - **highlight syntax with colors**
    - *even* **"debug"** *a program*
  - *The components of an IDE are designed to work together. For example,*
    - *when the compiler detects an error in a program, it can arrange for the editor to highlight the line that contains the error.*

There's a great deal of variation among IDEs, so I won't discuss them further in this book. However, I would recommend checking to see which IDEs are available for your platform.

Popular IDEs are:



2.2. General Form

```c
// hello.c

#include <stdio.h>

int main(void) {
    printf("hello, world\n");
}
```

```c
// general form

#preprocessor directives

int main(void) {
    statements
}
```

REYNEP's Note on BackStory

K.N. King's TextBook was written in **1996**. 2nd edition came out in 2008, and it's hard to change every single line of the book. So, I believe, the author kinda kept a few lines from 1996 here.

Back then, towards the end of **1980s**, in universities and high schools, for almost 70-80% of the courses, `Pascal` / `Turbo Pascal` was used! And the most popular / widely used programming languages were:

- `FORTRAN` *(1957)*,
- `COBOL` *(1960)*, `BASIC` *(1964)*
- `Pascal` *(1970)*, `Turbo Pascal` *(1983)*
- `Ada` *(1980)*, `SQL` *(1973)*

Now, except for FORTRAN, All of these kind of used `begin` & `end` instead of `{}` .

Now here's the text that the author wrote back in **1996** (I presume).

Notice how the `{}` braces show where `main()` **begins** and **ends**.
Now, **C** uses `{` and `}` in much the same way that some other languages use words like `begin` and `end` .

This illustrates a general point about **C**:

- ***C** relies heavily on **abbreviations** and **special symbols***
  - *which is one reason that C programs are concise (or—less charitably—cryptic).*
- *Even the simplest **C** programs rely on **3 key features**:*
  - *i.* **preprocessor-directives**: *(editing commands that modify our* `hello.c` *program prior to compilation)*
    - `#include <stdio.h>`
    - `#include` *is called a* **"directive"** / **"command"**
    - *"modify": as in* **"copy-paste"** *the entire content of a file called* **"stdio.h"**
  - *ii.* **functions**: *(named blocks of executable code)*
    - `int main(void)` ➡️ *here* `main` *is a function*
  - *iii.* **statements**: *(commands to be performed when the program is run).*
    - `printf("hello, world\n");` ➡️ *is a statement*

We'll take a closer look at these features now.

REYNEP's Note: Now, Shahriar Ivan kept these exact same lines, word by word in his slides. So i didn't modify/edit them.

2.2.1 General Form : `PreProcessor Directives`

Recall Back to 2.1.4. Summary `reynep's`

1. ***PreProcessing***
   - `#include` = **"directive"** / **"command"** - *(starts with* `#` *)*
   - `#include <stdio.h>`
     - **copy-pastes** *all the content of a file called* **"stdio.h"**
2. ***Compiling***
   - **Compiler** *translates* `hello.c` *into* **"binary machine instructions"**
     - *e.g.* `hello.c` ➡️ `hello.obj`
3. ***Linking***
   - **linker** *combines the* `hello.obj` *with say* `stdio.obj`
4. ***gcc***
   - `gcc` / `cc` / `clang` / `msvc` *does all 3 steps, then, after* **linking**, *hands us* `hello` / `hello.exe`

```
gcc hello.c -o hello         # Linux
gcc hello.c -o hello.exe     # Windows (available via, `msys2` + `mingw64`)
```

New in this Section

- `#include <stdio.h>`
  - *Before a* **C** *program is* **compiled**, *it is first edited by a* **preprocessor**.
    - *"edit": copy-paste/add/include*
  - *Commands intended for the preprocessor are called directives.*
    - *Chapters 14 and 15 discuss directives in detail.*
    - *For now, we're interested only in the* `#include` *directive.*
    - *e.g.*
      - `#include <stdio.h>`
  - `#include <stdio.h>`
    - **copy-pastes** *all the content of a file called* **"stdio.h"**
  - `<stdio.h>` *is called a* **"header file"**
    - *C has a number of* **"headers"**

```
Header
stdio.h    ➡️ printf(), scanf(), getchar() + more Input/Output stuffs.
stdbool.h  ➡️ bool, true, false
math.h     ➡️ sqrt(), pow(), sin(), cos()
ctype.h    ➡️ isdigit(), isalpha(), toupper(), tolower()
time.h     ➡️ time()
stdlib.h   ➡️ rand(), exit(), malloc(), free()
string.h   ➡️ strlen(), strcpy(), strcat()
... (more header-files)
```

      - *All of these header-files together is called* **"the standard library (STL)"**.
      - `<stdio.h>` *contains information about* **C's** *standard I/O library.*
  - **Preprocessor-Directives** *always begin with a* `#` *character.*
    - *By default, directives are 1 line long*
    - *There's no semicolon or other special marker at the end of a directive.*

2.2.2 General Form : `Functions`

Function is a named block of executable code. **Functions** are like “procedures” or “subroutines” in other programming languages (as of **1996**)—they’re the building blocks from which programs are constructed. In fact, a `C` program is little more than a collection of functions. **Functions** fall into 2 categories:

1. *Those written by the programmer / User defined Functions*
2. *Those provided as part of the `C` implementation ➡️ **"library functions"**, since they belong to a **"library"** of functions that are supplied with the **compiler**.*

- ***Functions***
  - *Introduced in Chapter 09*
  - `int main(void)` *is a function*
    - *name of this specific function :* `main`
  - *The term “function” comes from mathematics:*
    - $f(x) = x + 1$
  - *A function that computes a value uses the return statement to specify what value it* **"returns."**
    - *(very dumb line to say right here)*
    - *we must check out Chapter 09, to actually understand* **"return"**

2.2.3 General Form : The `main` function

- *The `main` function is Mandatory*
  - *The name `main` is critical.*
    - *It can't be anything else like `whatever`*
    - *It can't even be `MAIN` or `mAiN`*
    - *It must be alphabetically `main`*
  - *Because, the people who made the* **compiler** *decided, that, `main()` is where the program should start. So like, when we run `hello`, Statements from inside `main()` starts to be executed one by one.*
- `return 0;`
  - *This won't be fully clear before* **Chapter 9**
  - *(but, Ivan kept 2 lines here.)*
  - `return 0;` *means, The Status Code "0" is handed back to the Terminal/Command-Line @ the end of the execution. 1980s convention was that people used to* `return 404;` *or say* `return -1;` *to mean certain kinds of errors.* `return 0` *means, everything is okay, normal program execution.*
  - *Yes,* `return 0;` *inside* `main()` *is not hardcode required.*
    - *But, many* **compilers** *will produce warning messages if no* `return` *statement is provided at the end of* `main()` **function**.

2.2.4 General Form : `Statements`

A **statement** is a **"command"** to be executed **"when the program runs"**. (unlike **preprocessor-directives/commands**). We'll explore statements later in the book, primarily in **Chapters 5 and 6**.

The `hello.c` program uses only two kinds of statements:

1. *the* `return` *statement*
2. *the* ***function*** *call.*

```
printf("Hello, World\n");
```

- *Asking a function to perform its assigned task is known as* **"calling the function"**.

`C` requires that each statement end with a **semicolon**.

- *exception:* ***compound statement***
  - *e.g.*
- *The* ***semicolon*** *shows the compiler where the statement ends;*

2.2.5 General Form : `Printing Strings`

`printf` is a powerful **function** that we'll examine in **Chapter 3**.
So far, we've only used `printf` to display a **"string literal"**—a series of characters enclosed in double quotation marks. When `printf` displays a string literal, it doesn't show the quotation marks.

`printf` doesn't automatically advance to the next output line when it finishes printing.

```
printf("Hello ");
printf("World\n");
    // output:
    //     Hello World      (1 line)
```

To instruct `printf` to advance one line, we must include `\n` (the new-line character) in the string (i.e. inside Quotation marks). Writing a new-line character terminates the current output line; subsequent output goes onto the next line.

2.3 Comments

- *Can appear in anywhere in the program*
- *Comments may extend over more than one line*

- ***MultiLine Comment (C89 Style)***
  - *begins with* `/*` *and ends with* `*/`
  - *Multiline comments stands out better than* **Single-Line** *(see below) Comment*

```
/*
    Name: hello.c
    Purpose: Prints Hello, world
    Author: 🤷
*/

#include <stdio.h>
int main(void)
{                               /* beginning */
    printf("Hello, world\n");   /* print function */
    return 0;
}
```

  - *Failure to end a comment can result in program* **Statements** *being* **commented out** *causing your program to be incorrect.*

```
/*
    Name: hello.c
    Purpose: Prints Hello, world
    Author: 🤷

#include <stdio.h>
int main(void)
{                               /* beginning */
    printf("Hello, world\n");   /* print function */
    return 0;
}
```

- ***Single Line Comment (C99 Style)***
  - *begins with* `//`

```
//  Name: hello.c
//  Purpose: Prints Hello, world
//  Author: 🤷

#include <stdio.h>
int main(void)
{                               // beginning
    printf("Hello, world\n");   // print function
    return 0;
}
```

  - *we don't specify smth, to mean end of comment.*
    Simply, everything to the right-side of `//` *will be* **"comment"**.
    - *Thus,* **Less error prone** *: no chance of unterminated comment to consume part of a program*
    - *Simple and helpful because it ends automatically at the end of the line*

2.4. Variables and Assignment

2.4.1. `Types`

Few programs are as simple as the `hello.c` . Most programs need to perform a series of calculations before producing output, and thus need a way to store data temporarily during program execution. In `C` , as in most programming languages, these storage locations are called **variables**.

```
int x = 10;
float y = 5.5;
```

Every variable must have a `type` , which specifies what kind of data it will hold. `C` has a wide variety of `types` .
For now, we'll limit ourselves to just two: `int` *and* `float` .

- `int`
  - *Short for* **"integer"**
  - *can store a whole numbers such as 0, 1, 392, or –2553*
  - *Limited Range*
    - *Largest:* `2,147,483,647`
- `float`
  - *Short for* **"floating point numbers"**
  - *can store (theoretically) much larger decimal numbers than* `int`
  - *can store numbers with digits after the decimal point, like 379.125*
  - *Limitations:*
    - *Slower Arithmetic Operations (compared to* `int` *)*
    - *Round-off error*
      - *e.g. you can't store* `0.1000000159` *or stuff like that*
      - *instead* `0.100000016391277313232421875` *is stored*
      - *So,* `float` *has a nature of like, "Approximation"*

2.4.2. `printf`

- `printf` `ivan's lines`
  - `printf` *can be used to display the current value of a variable*

```
int h = 70;                    // say 70inches = 5feet10inches
printf("Height: %d\n", h);
    // outputs:
    //      Height: 70
```

  - `%d` *is a placeholder indicating where the value of height is to be filled in*
  - `%d` *works for* `int`
  - `%f` *is required for* `float`
    - *By default,* `%f` *displays* **six digits after the decimal points**

```
int profit = 2150.480000111f;
printf("Profit:   $ %f", profit);
printf("\n");
    // outputs:
    //      Profit:   $ 2150.480000
```

```
int profit = 2150.480000111f;
printf("Profit:   $ %.2f", profit);
printf("\n");
    // outputs:
    //      Profit:   $ 2150.48
```

    - *To force* `%f` *to display* `k` *digits after the decimal point, put* `.k` **between** `%` *and* `f`

  - *multiple variables inside* **single call** *of* `printf` *:*

```
int height = 8;                        int length = 12; int width = 10;
printf("Height: %d Length: %d\n", height, length);
```

    - *There's no limit to the number of variables that can be printed by a single call of printf:*

2.4.3. `Declarations`

```
int main(void)
{
    declarations
    statements
}
```

```
int main(void)
{
    int height = 8;
    int length = 12;
    int width = 10;
    float profit;

    int volume = height * length * width;
    printf("Volume: %d\n", volume);
    return 0;
}
```

- `Variables`
  - *must be declared before they are used*
  - *can be declared one at a time*

```
int height;
float profit;
```

  - *several can be declared in the same line*

```
int height, length, width, volume;
float profit, loss;
```

- `Assignment`
  - *is the process of assigning a value to a variable*

```
height = 8;
profit = 100.0;
```

  - *declare first, then assign*

```
// Correct way
int height;
    height = 8;
```

```
// Wrong way
height = 8;
int height;
```

  - *multiple variable declaration in 1 line:* caveat

```
// Prefferred way
int length = 12, width = 10;
```

```
// Not wrong, but be careful
int length, width = 10;
```

  - *floating point number assignments*
    - *It's best to append the letter f to a floating point constant*

```
float profit = 100.0f;
```

  - *Assigning* `int` *inside* `float` */ or / the opposite*

```
float y = 5;        // REYNEP: meh, it's okay 🤷
int   x = 5.7;      // REYNEP: 5 will be stored inside 'x'. Decimal part is thrown away.
```

    - *ivan: Assigning* `int` *to float, or vice versa is possible but not recommended*

- `Operations` *& More* `Assignments`
  - *Once* **assigned**, *a* **variable** *can be used to* **compute** *the value of* **another variable**:

```
int height = 10, length = 20, width = 5, volume;
volume = height * length * width;                    // volume is now 1000
```

  - *The right side of an assignment can be a* **"formula"**
    - *involving* **constants**, **variables**, *and* **operators** *(* `+` `-` `*` `/` `%` *)*
    - **"formula"**
      - *is called* **"expression"** *in* `C` *terminology*

- `Uninitialized Variables`
  - *Some variables are automatically set to 0, when a program begins to execute, but most are not.*
  - *A variable that doesn't have a default value and hasn't yet been assigned a value by the program is said to be uninitialized.*

```
int height;
    // declared   but   uninitialized variable
int width = 50;
    // declared   and     initialized variable
```

  - *Attempting to access the value of an* **uninitialized variable**
    - *e.g. by displaying the variable using printf*
    - *using it in an expression*
  - *may yield an unpredictable result such as 2568, –30891, or some equally strange number.*
    With some compilers, worse behavior—even a program crash—may occur.

```
int height;
printf("%d", height);
    // might print 2568, might print -30891, might print 0.... so on.... no pattern
```

- `Operations`*:* `Caveat #1`

```
float x = 5/9;          // stores 1
float x = 5.0/9.0;      // stores 0.555555
float x = 5.0f/9.0f;    // stores 0.555555
```

2.4.4. Problem: `Computing the Dimensional Weight of a Box`

Shipping companies don't especially like boxes that are large but very light, since they take up valuable space in a truck or airplane. In fact, companies often charge extra for such a box, basing the fee on its volume instead of its weight. In the United States, the usual method is to divide the volume by 166 (the allowable number of cubic inches per pound). If this number—the box's "dimensional" or "volumetric" weight—exceeds its actual weight, the shipping fee is based on the dimensional weight. (The 166 divisor is for international shipments; the dimenional weight of a domestic shipment is typically calculated using 194 instead.)

Let's say that you've been hired by a shipping company to write a program that computes the dimensional weight of a box. Since you're new to `C`, you decide to start off by writing a program that calculates the dimensional weight of a particular box that's `12" × 10" × 8"`. Division is represented by `/` in `C`, so the obvious way to compute the dimensional weight would be `weight = volume / 166;`

where weight and volume are integer variables representing the box's weight and volume. Unfortunately, this formula isn't quite what we need. In `C`, when one integer is divided by another, the answer is "truncated": all digits after the decimal point are lost. The volume of a `12" × 10" × 8"` box will be 960 cubic inches. Dividing by 166 gives the answer 5 instead of 5.783, so we have in effect rounded down to the next lowest pound; the shipping company expects us to round up. One solution is to add 165 to the volume before dividing by 166: `weight = (volume + 165) / 166;`

A `volume of 166` would give a weight of `331/166`, or `1`, while a `volume of 167` would yield `332/166`, or `2`. Calculating the weight in this fashion gives us the following program.

```c
/* dweight.c */
/* Computes the dimensional weight of a 12" x 10" x 8" box */
#include <stdio.h>

int main(void)
{
    int height, length, width, volume, weight;

    height = 8;
    length = 12;
    width = 10;
    volume = height * length * width;
    weight = (volume + 165) / 166;
        // why? Read problem statement
        // here's a TL;DR tho
        // volume = 166   ==>   weight = 331/166 = 1
        // volume = 167   ==>   weight = 332/166 = 2

    printf("Dimensions: %dx%dx%d\n", length, width, height);
    printf("Volume (cubic inches): %d\n", volume);
    printf("Dimensional weight (pounds): %d\n", weight);

    return 0;
}
```

The output of the program is

```
Dimensions: 12x10x8
Volume (cubic inches): 960
Dimensional weight (pounds): 6
```

2.5. Reading Input

```c
#include <stdio.h>

int main(void)
{
    int x = 0;
    float y = 0;
    scanf("%d", &x);
    scanf("%f", &y);

    printf("Your Integer: %d", x);
    printf("Your   Float: %f", y);
}
```

- `scanf` *is the* `C` *library's counterpart to* `printf()`
- `scanf` *requires a* ***"format string"*** *to specify the appearance of the input data*
  - *a.k.a* `%d`
- *the* `&` *symbol is required when using* `scanf()`
- *We can modify our code from last page to take inputs from user using* `scanf()`

```c
    /* dweight.c */
    int height, length, width, volume, weight;

    printf("Enter height of the box: ");
    scanf("%d", &height);
    printf("Enter length of the box: ");
    scanf("%d", &length);
    printf("Enter width of the box: ");
    scanf("%d", &width);

    volume = height * length * width;
    weight = (volume + 165) / 166;
```

  - *output*

```
Enter height of the box: 8
Enter length of the box: 12
Enter width of the box: 10
Volume (cubic inches): 960
Dimensional weight (pounds): 6
```

2.6. Defining Macros

```
#define RECIPROCAL_OF_PI (1.0f / 3.14159f)

#define INCHES_PER_POUND 166
#define ROUND_UP_OFFSET INCHES_PER_POUND-1

weight = (volume + ROUND_UP_OFFSET) / INCHES_PER_POUND;
```

- `#define` *is a* ***preprocessing directive****,*
  - *just as* `#include` *was*
  - *so there's no semicolon at the end of the line.*
  - *When a program is* **compiled***, the* **preprocessor** *replaces each* **"macro"** *by the value that it represents.* **e.g.**

    ```
    weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
        // becomes
    weight = (volume + 166 - 1) / 166
        // after the preprocessing round
    ```

  - *The value of a macro can be an expression:*

    ```
    #define RECIPROCAL_OF_PI (1.0f / 3.14159f)
    ```

    - *If it contains operators, the expression should be enclosed in parentheses.*

When a program contains constants, it's often a good idea to give them names. The `dweight.c` and `dweight2.c` programs rely on the `constant 166`, whose meaning may not be at all clear to someone reading the program later. Using a feature known as **"macro definition"**, we can name this constant: `#define INCHES_PER_POUND 166`

Notice that we've used only upper-case letters in macro names. This is a convention that most `C` programmers follow, not a requirement of the language. (Still, `C` programmers have been doing this for decades; you wouldn't want to be the first to deviate.)

2.6.1. Problem: `Converting from Fahrenheit to Celsius`

The following program prompts the user to enter a Fahrenheit temperature; it then prints the equivalent Celsius temperature. The output of the program will have the following appearance (as usual, input entered by the user is underlined):

```
Enter Fahrenheit temperature: 212
Celsius equivalent: 100.0
```

The program will allow temperatures that aren't integers; that's why the Celsius temperature is displayed as 100.0 instead of 100. Let's look first at the entire program, then see how it's put together.

```c
/** celcius.c */
/* Converts a Fahrenheit temperature to Celsius */
#include <stdio.h>

#define FREEZING_PT 32.0f
#define SCALE_FACTOR (5.0f / 9.0f)

int main(void)
{
    float fahrenheit, celsius;
    printf("Enter Fahrenheit temperature: ");
    scanf("%f", &fahrenheit);

    celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;

    printf("Celsius equivalent: %.1f\n", celsius);
    return 0;
}
```

The statement `celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;` converts the Fahrenheit temperature to Celsius. Since `FREEZING_PT` stands for `32.0f` and `SCALE_FACTOR` stands for `(5.0f / 9.0f)`, the compiler sees this statement as:

```c
celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;    // Before PreProcessing
celsius = (fahrenheit - 32.0f)       * (5.0f / 9.0f);         // After PreProcessing
```

Caveats

```c
#define SCALE_FACTOR (5.0f / 9.0f)
```

- *Defining* `SCALE_FACTOR` = `(5.0f / 9.0f)` *instead of* `(5 / 9)` *is important, because* `C` ***"truncates"*** `5/9` *into just* `1` *a.k.a* ***"truncated"*** *when two integers are divided.* ***"truncation"*** *is kinda like* ***mathematical "floor"ing***

```c
printf("Celsius equivalent: %.1f\n", celsius);
```

- *Notice the use of* `%.1f` *to display celsius with just one digit after the decimal point.*

2.7. Identifiers 🟩 `ivan's lines`

- *Names of **variables**, **functions**, **macros** (+ a few more other entities) are called **"identifiers"***

- 🎄 *An **"identifier"** may contain* `letters` , `digits` , *and* `underscores` .
- 🎄 *An **"identifier"** must begin with a* `letter` / `underscore` .
  - `times10`
  - `get_next_char`
  - `_done`

- 🟥 *An **"identifier"** **cannot** begin with a digit.*
  - `10times`
  - `get-next-char`
    - *these are* **illegal "identifiers"**

- *C places no limit on the maximum length of an identifier*

- `C` *is* `case-sensitive`
  - `C` *distinguishes between upper-case and lower-case letters in* **identifiers**
  - *e.g.*
    - `job` != `joB` != `jOb` != `jOB` != `Job` != `JoB` != `JOb` != `JOB`
    - *all of them identifiers are all different*

  - `SnakeCase`
    - *Many programmers use only lower-case letters in* **identifiers**, *with underscores inserted for legibility*
    - `symbol_table`
    - `current_page`
    - `name_and_address`
    - *(other than macros)*

  - `CamelCase`
    - *Other programmers use an upper-case letter to begin each word within an identifier:*
    - `symbolTable`
    - `currentPage`
    - `nameAndAddress`

- *The following* `keywords` *can't be used as **identifiers**:*

```
auto      enum      restrict*  unsigned
break     extern    return     void
case      float     short      volatile
char      for       signed     while
const     goto      sizeof     _Bool*
continue  if        static     _Complex*
default   inline*   struct     _Imaginary*
do        int       switch
double    long      typedef
else      register  union
```